Ports

In Assembly, we have two instructions:

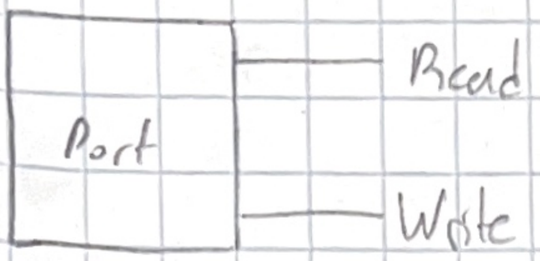
Writing: out

Reading: in

I believe these instructions are specifically used for x86 CPU architectures.

Essentially, we write and read from these ports when certain actions occur, and this is the main way our C++ Kernel communicates to the CPU.

In our program, we don't want to use Assembly, so we will abstract Ports so it abides by our OOP principles.



There are multiple port sizes:
inb/outb: 8 bits (1 byte)
inw/outw: 16 bits
inl/outl: 32 bits

We can send lower bit data to larger bit ports, (due to sign extension), but not the other way around (compiler will through error catching it for us)

We may also need "slow" ports by jumping lines to make the program wait a little until port is done writing the data.

This is needed for "classical controllers" such as the keyboard, P.I.C.

Global Descriptor Table

The Global Descriptor Table is a data structure that contains entries telling the CPU about memory segments in the computer's memory.

In my implementation, the GDT has three Segment Descriptors:

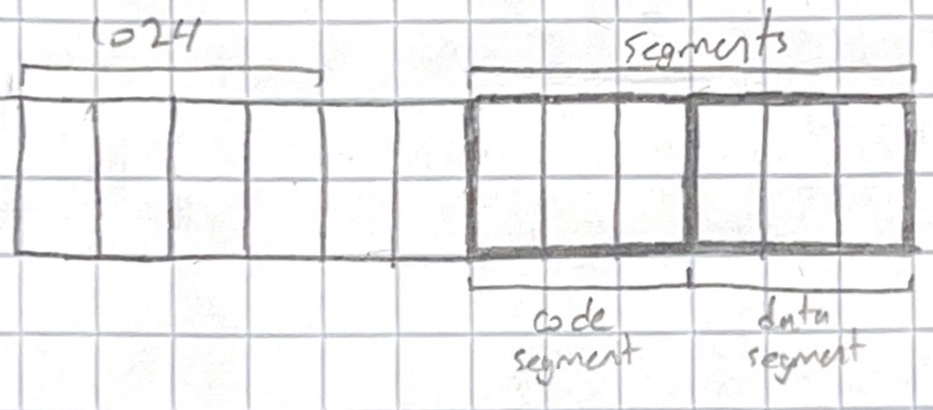
① Null Segment Selector
- Located at entry 0 in the descriptor table
- Never referenced by the processor and may be required on certain CPUs or emulators
- Usually used to indicate a Null value or error

② Unused Segment Selector
- Left unused in this Kernel

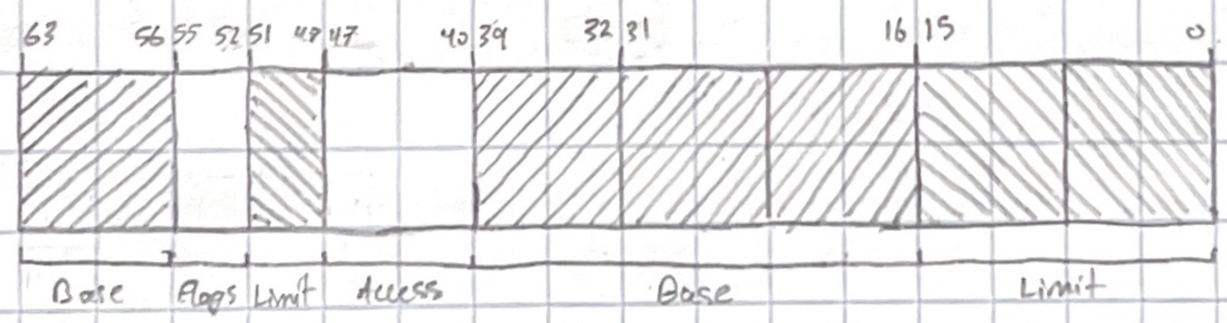
③ Code Segment Descriptor
- The location in memory where the Kernel's executable code is located

④ Data Segment Descriptor
- The location in memory where the Kernel can read and write data

In memory, this would look like:



Each entry in the table is 8 bytes



Base (pointer): 4 bytes / 32 bits
Limit: 2.5 bytes / 20 bits

The reason they are clustered is because the entry was extended (history) and for backwards compatibility reasons.

Segment Descriptors

Segment descriptors describe a memory segments base address, limit (size) and access rights (Flags).

In the code, we initialized a segment desciptor by first getting a panter to the current segment descriptor object in memory:

```
uint8_t* target = (uint8_t*)this;
```

Then, we alter the bits of the target so that it is filled with the information for the base, limit and flags.

Since the target is 8 bit, it can be treated as an array with length 8 with each element representing a byte, or 8 bits:

```
target[0] = limit & 0xFF;          )→ bits 15 to 0
target[1] = (limit >> 8) & 0xFF;   )
target[6] = (limit >> 16) & 0xF;   )→ bits 51 to 48

target[2] = base & 0xFF;           )
target[3] = (base >> 8) & 0xFF;    )→ bits 39 to 16
target[4] = (base >> 16) & 0xFF;   )
target[7] = (base >> 24) & 0xFF;   )→ bits 63 to 56

target[5] = Flags                  )→ bits 47 to 40
```

When we instantiate the GDT object, we set the address of the GDT object and the the size of the GDT object in the variable "i", then load this information into the GDT register using Assembly:

```
uint32_t i[2];
i[1] = (uint32_t)this;
i[0] = sizeof(GlobalDescriptorTable) << 16;
asm volatile("lgdt (%0)" :: "p"(((uint8_t*)i)+2));
```

Interrupts

Interrupts are signals sent to the CPU indicating that an event needs immediate attention.

When an interrupt arises, the CPU looks up an entry for that specific interrupt from a table provided by the OS. This is called the interrupt descriptor table, and it can have up to 256 entries.

Once the CPU finds the entry for the interrupt, it jumps to the code the entry points to. (interrupt handler) and runs it.

Types of interrupts:

① Exceptions: Internally generated by the CPU and used to alert the kernel of an event that needs to be addressed

② Interrupt Request/Hardware Interrupt: Generated externally

③ Software Interrupts

Hardware

When a piece of hardware emits an interrupt, for example a key press on a Keyboard, it tells the Programmable Interrupt Controller (PIC) to cause an interrupt.

The OS handles the interrupt by talking to the Keyboard via ports (in/out instructions) to ask what key was pressed.

IRQs are signals sent to the PIC indicating what type of interrupt it is. These are standards that most PCs follow:

IRQ (decimal/hex)	Description
0/0x00	Timer
1/0x01	Keyboard
12/0x0C	Mouse

Programmable Interrupt Controller (PIC)

Most systems have two PICs: slave and master, each with 8 input signals and 1 output signal used to tell the CPU that an IRQ ocurred.

The slave's output is connected to the master's third input (#2). This takes up the spot for IRQ #2, which is known as the "cascade" interupt.

In the code, an `InterruptManager` class handles all of this:

picMasterCommand(0x20) } → initialize master PIC port
picMasterData(0x21)
picSlaveCommand(0xA0) } → initialize slave PIC port
picSlaveCommand(0xA1)

After this, we initialize a pointer to the code segment from the GDT, and we also set all 256 interrupt entries to be ignored.

* Then, using the `SetInterruptDescriptorTableEntry` method, we add the interupt handlers for times, keyboard and mouse interrupts (+ other ones in the future).

Then, we run the PIC initialization sequence, and finally, load the interrupt descriptor table using the `lidt` assembly instruction.

* When we add the interupt handlers in the Kernel using C++, we are referencing what seems to be uninitialized methods. These methods are actually connected to the hardware using interruptstubs.s.

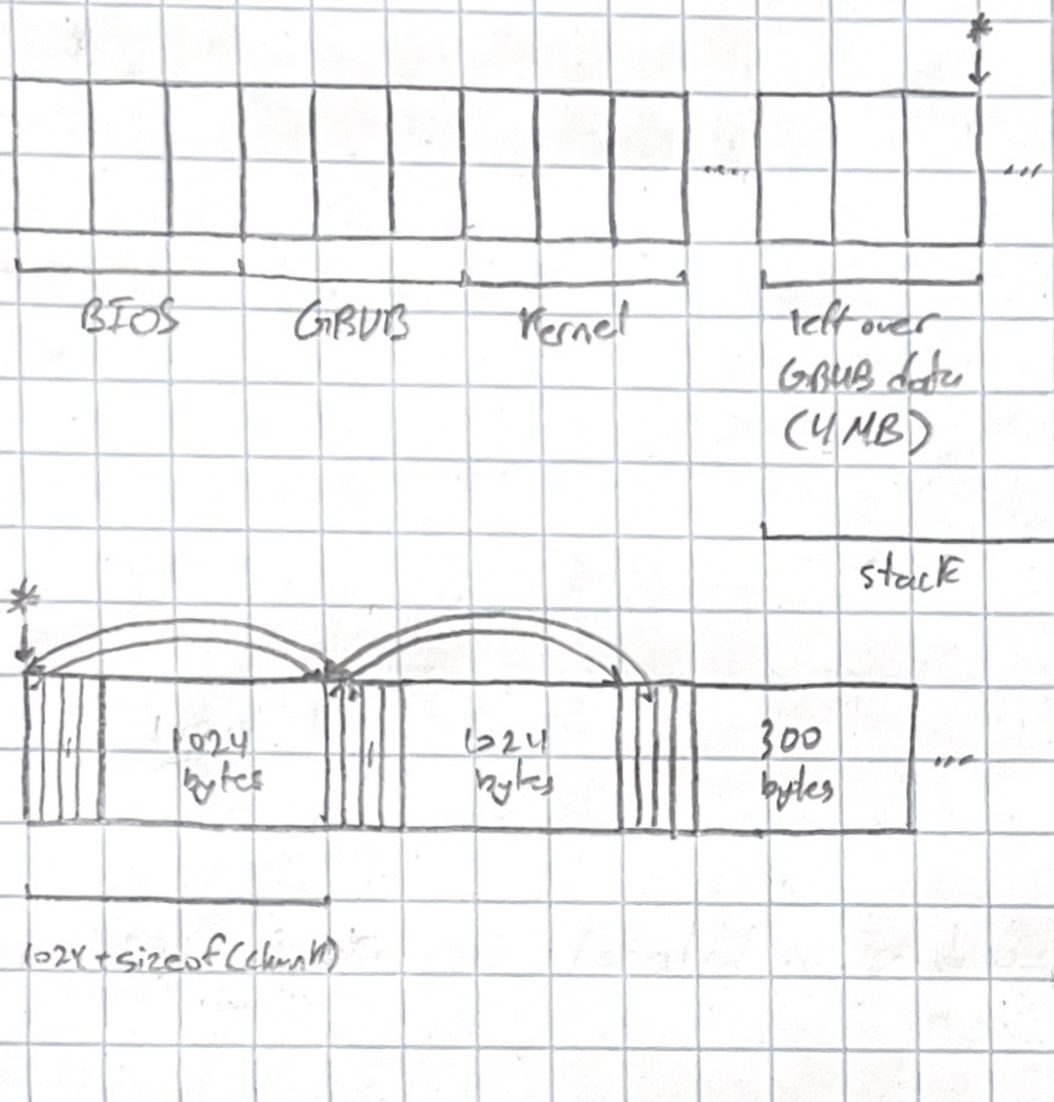
Then, the CPU calls `handleInterrupt` using an interruptNumber parameter, which calls the interrupt handler's handle method in interruptManager→handlers[#]. If you trace the codes you will see what I mean.

These handlers are added when we add drivers in the Kernel, which you will see later

Memory Management

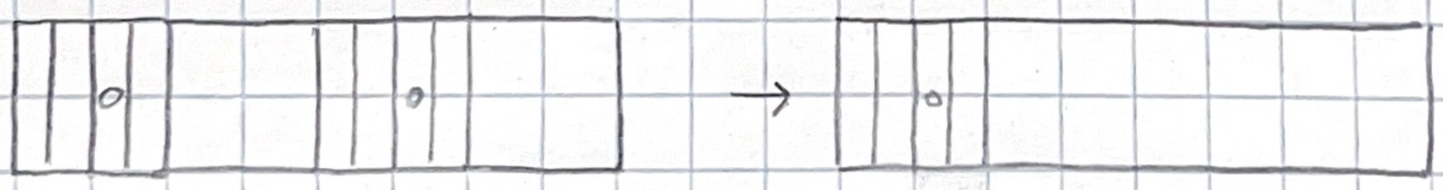
Memory Management in a Kernel provides a way for programs to allocate and free memory.

In the code, we have a **Memory Chunk** structure that is stored within the heap region in RAM. Each Memory Chunk has a pointer to the previous chunk, next chunk, allocated flag and size of chunk excluding the header:



The next and previous pointers point to adjacent memory chunks. The allocated flag indicates if the chunk is being used or not. The size of the chunk indicates the size of the data of the memory chunk.

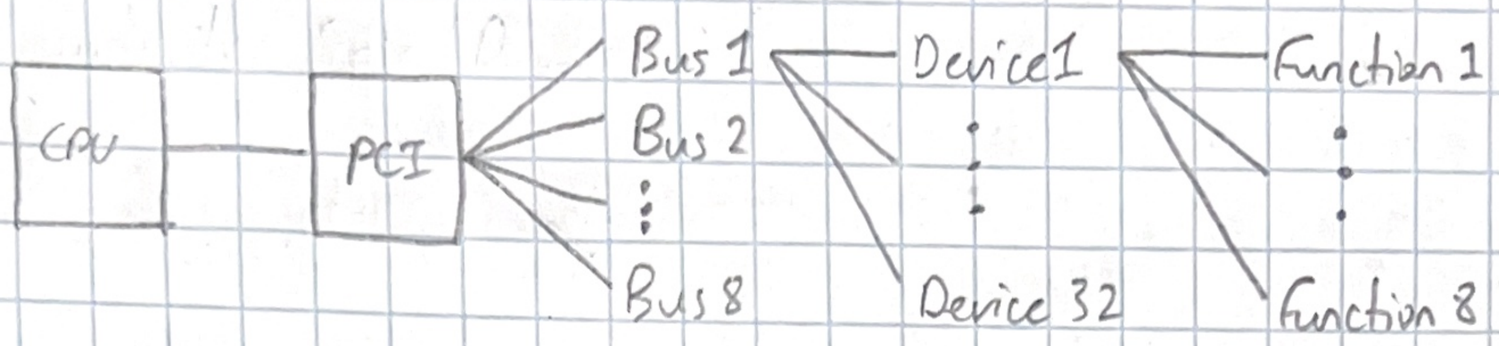
When we clear multiple adjacent chunks, we want to merge them so that don't have adjacent chunks with small sizes.



In the memory, we start our memory management at 10MB and pass the size of the entire dynamic memory area as the total size - start.

Peripheral Component Interconnect

This is a standard for connecting peripheral devices, like network cards, sound cards, and more to a computer's motherboard.



The PCI has 8 buses, each with 32 devices, each with up to 8 functions.

In our code, we "brute force" through all Buses, Devices and functions and at the time of writing this, only prints the details.

We get the details from the `GetDeviceDescriptor` method, which returns a device descriptor structure that has the bus #, device #, function #, as well as other details defined by an offset:

Field	Offset	Description
vendor_id	0x00	Manufacturer ID
device_id	0x02	Device Model ID
⋮	⋮	⋮
interrupt	0x3c	Interrupt line used by device

Once we get the device's information, we check if it is valid (!=0x0000 \ !=0xFFFF) based on it vendor id.

Then, once we get it as a driver, if it is defined in our system, and add it to our driver manager.

The drivers themselves are used to set up certain hardware.

Graphics

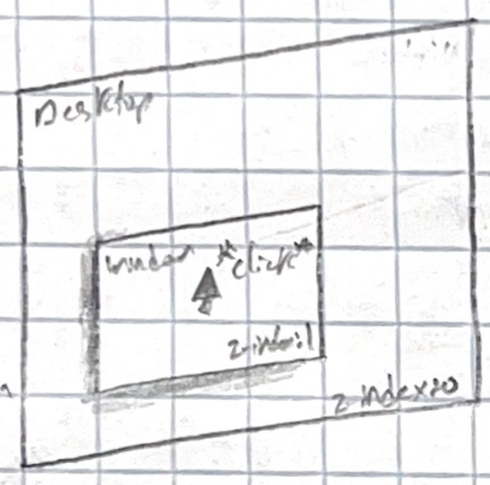
Graphics Mode is a different mode for operating systems. We can use this mode to display the desktop and windows.

Currently, this project doesn't dive too deep into GUI, but it covers the basics

The desktop and windows are what are known as widgets. Widgets can be be drawn using the `fillRect` method from the VGA driver.

They also handle various events such as focus, onMouseDown, onMouseUp.

On layered widgets, events on a specific widget is determined based on their z-index:



Currently, rendering occurs on the CPU, which is slow. Eventually, more graphics features may be added.

Graphics

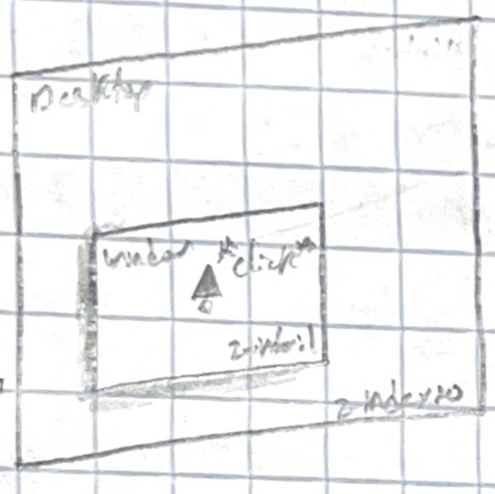
Graphics Mode is a different mode for operating systems. We can use this mode to display the desktop and windows.

Currently this project doesn't dive too deep into GUI, but it covers the basics

The desktop and windows are what are known as widgets. Widgets can be be drawn using the [fillRect] method from the VGA driver.

They also handle various events such as focus, onMouseDown, onMouseUp.

On layered widgets, events on a specific widget is determined based on their z-index:



Currently, rendering occurs on the CPU, which is slow. Eventually, more graphics features may be added.